6.4.1. Compose

Agtron has to pay tax (that's one thing you don't need to imagine). To calculate the tax, you first need to add up the profit for all of the products and then invoke a `tax` function with that value.[1] If you did this imperatively (with commands), you'd make all the individual calculations in sequence:

[1] In practice, the arithmetic operators in JavaScript and CoffeeScript aren't good when working with decimal values because JavaScript's numbers use floating-point arithmetic, which isn't accurate for decimal values. If you need to do arithmetic with decimal values, you should find an appropriate library for decimal arithmetic.

```
profit = ->

tax = (amount) ->
  amount / 3

netProfit = (products) ->
  profits = (profit product) for product in products
  profits.reduce (acc, p) -> acc + p

netProfitForProducts = netProfit products
taxForProducts = tax netProfitForProducts
```

Omitted. The profit function appears elsewhere.

See the footnote about floating-point arithmetic.

Imagine now that you also need to evaluate a loyalty discount for a user. To do this imperatively, you first work out the total amount they've spent and then use that value to determine the loyalty category:

```
userSpend = (user) ->
  spend = 100

loyaltyDiscount = (spend) ->
  if spend < 1000 then 0
  else if spend < 5000 then 5
  else if spend < 10000 then 10
  else if spend < 50000 then 20
  else if spend > 50000 then 40

fredSpend = userSpend fred
loyaltyDiscountForFred = loyaltyDiscount fredSpend
```

Function abstraction is about identifying patterns. What's the pattern common to determining tax and determining a loyalty discount? They both